



Name : Deras Pangestu Gusti (08)

Class : SIB 1G

NIM : 254107060114

JOBSHEET

Case Study 2

1. Objective

- Students are able to solve case study problems by applying the concept of Linear Data Structures.
- sorting and searching algorithms according to case studies.

2. Practicum

Processing Time: 200 minutes

Work Instructions:

Each student **will code a program** based on a given case study. Ensure the program runs smoothly, implements all requested features, and is dynamic (using menus to select features).

Case Study Questions:

At lunch time, the Royal Delish restaurant has many customers coming at the same time. Cashiers have difficulty recording customer queues and food orders because the process is still done manually.

As a result:

- Queue numbers are often mixed up.
- Buyer data is not recorded neatly.
- Food orders are hard to recap.

To overcome this problem, a **Linked- based queue information system was created. List / Double Linked List**.

System flow:

- Buyers come and take a queue number.
- Buyer data is entered into the system.
- When called, the customer places a food order.
- Order data is saved.
- The system can display order reports and total restaurant revenue.

In this system there are two main classes , namely:

Buyer	Order
Name : String	OrderCode : int
MobileNumber : String	OrderName : String
Buyer(String b, String c)	price : int
	Order(int a, String b, int d)

The image above is an example of a class diagram of Buyers and Orders. Attributes and methods can be added.

Must-Have Features:

1. Add Queue

When you select the "Add Queue" menu, the system will add a queue number according to the existing queue order. Queue numbers can be generated automatically. The system will then ask for input, including the customer's name and phone number. The entered data will be saved in the linked queue. list.

Example:

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 1
Buyer Name : Zaki
No HP      : 09876543
Antrian berhasil ditambahkan dengan nomor: 4
=====
```

Note: Use data addition by creating objects using constructors to avoid time-consuming data input via keyboard .

2. Print Queue

This menu will display all the queues that are on linked list

Example :



```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 2
=====
Daftar Antrian Pembeli
=====
```

No Antrian	Nama	No HP
1	Ainra	08224500000
2	Danra	08224511111
3	Sanri	08224522222
4	Zaki	09876543

3. Clear Queue

Clear the queue must match the queue number. The clear queue menu indicates the customer has gone to the cashier to order food. Therefore, the customer's food order must be entered in this menu.

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 3
Order Code : 123
Order Name : Orange Juice
Price      : 25000
Ainra telah memesan Orange Juice
```

then the queue must be reduced. In the example below , there are already 3 customers who have ordered food so there are 1 people left in the queue



```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 2
=====
Daftar Antrian Pembeli
=====
```

No Antrian	Nama	No HP
2	Danra	08224511111
3	Sanri	08224522222
4	Zaki	09876543

4. Order report

This menu displays all incoming orders. Order data should be displayed sorted by order name.

```
=====
ROYAL DELISH QUEUE SYSTEM
=====
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu : 4
=====
LAPORAN PESANAN (URUT NAMA PESANAN)
=====
```

Kode Pesanan	Nama Pesanan	Harga
123	Orange Juice	25000



Assessment Rubric

Components: Code Assessment: 30% + CM Practical Test Assessment: 70%

Code Assessment Rubric

No	Aspect Evaluation	Weight	Score 1	Score 2	Score 3
1	Implementation Queue Data Structure	20%	Do not use linked lists	Using Single Linked List	Using Double Linked List with Correct
2	Implementation Order Data Structure	20%	Do not use linked lists	Using Single Linked List	Using Double Linked List with Correct
3	Added Features Queue	10%	Not running	Walk part	Walk with Good
4	Print Features Queue	10%	Not showing	Showing some data	Showing all data with Correct
5	Queue & Order Input Features	15%	Not running	Walk part	Walk in accordance question
6	Report Order & Sorting	20%	No sorting	Sorting less appropriate	Sorting in progress Correct

Additional Terms

Programs may not use Collections Frameworks such as:

- ArrayList
- Linked List
- Queue
- Stack
- Collections.sort ()
- Sorting is done manually.

The program must use objects and constructors .

Double Use Linked The list in both data structures will get maximum value in the data structure aspect.

```
1 package com.cm2;
2
3 public class Buyer
4 {
5     int    queueNumber;
6     String name,
7         mobile;
8
9     Buyer (int q, String n, String m)
10    {
11        queueNumber = q;
12        name        = n;
13        mobile      = m;
14    }
15 }
```

```
1 package com.cm2;
2
3 public class Order
4 {
5     int    orderCode,
6         price;
7     String orderName;
8
9     Order (int c, String n, int p)
10    {
11        orderCode = c;
12        orderName = n;
13        price     = p;
14    }
15 }
```

```
1 package com.cm2;
2
3 public class NodeBuyer
4 {
5     Buyer    data;
6     NodeBuyer next,
7         prev;
8
9     NodeBuyer (Buyer b)
10    {
11        data = b;
12        next = null;
13        prev = null;
14    }
15 }
```

```
1 package com.cm2;
2
3 public class NodeOrder
4 {
5     Order    data;
6     NodeOrder next,
7             prev;
8
9     NodeOrder (Order o)
10    {
11        data = o;
12        next = null;
13        prev = null;
14    }
15 }
```

```
1 package com.cm2;
2
3 public class QueueBuyer
4 {
5     NodeBuyer head,
6             tail;
7     int size = 0;
8
9     boolean isEmpty()
10    {
11        return head == null;
12    }
13
14     void enqueue (Buyer b)
15    {
16        NodeBuyer newNode = new NodeBuyer(b);
17
18        if (isEmpty())
19            head = tail = newNode;
20        else
21        {
22            tail.next = newNode;
23            newNode.prev = tail;
24            tail = newNode;
25        }
26
27        size++;
28        System.out.println("Queue successfully added with number: " + b.queueNumber);
29    }
30 }
```

```
31     Buyer dequeue()
32     {
33         if (isEmpty())
34         {
35             System.out.println(x: "Queue is empty!");
36             return null;
37         }
38
39         Buyer removed = head.data;
40         head = head.next;
41
42         if (head != null)
43             head.prev = null;
44         else
45             tail = null;
46
47         size--;
48         return removed;
49     }
50
51     void printQueue()
52     {
53         if (isEmpty())
54         {
55             System.out.println(x: "Queue is empty!");
56             return;
57         }
58
59         System.out.println(x: "List of Buyers in Queue:");
60
61         NodeBuyer tmp = head;
62
63         while (tmp != null)
64         {
65             System.out.println(tmp.data.queueNumber + " | " + tmp.data.name + " | " + tmp.data.mobile);
66             tmp = tmp.next;
67         }
68     }
69 }
```

```
1  package com.cm2;
2
3  public class OrderList
4  {
5      NodeOrder head,
6      tail;
7
8      boolean isEmpty()
9      {
10         return head == null;
11     }
12
13     void addOrder (Order o)
14     {
15         NodeOrder newNode = new NodeOrder(o);
16
17         if (isEmpty())
18             head = tail = newNode;
19         else
20         {
21             tail.next      = newNode;
22             newNode.prev   = tail;
23             tail           = newNode;
24         }
25     }
26
27     void insertionSortOrders()
28     {
29         if (isEmpty())
30             return;
31
32         NodeOrder i = head.next;
33
34         while (i != null)
35         {
36             Order key = i.data;
37             NodeOrder j = i.prev;
38
39             while (j != null && j.data.orderName.charAt(index: 0) > key.orderName.charAt(index: 0))
40             {
41                 j.next.data = j.data;
42                 j           = j.prev;
43             }
44         }
```

```
45         if (j == null)
46             head.data = key;
47         else
48             j.next.data = key;
49
50         i = i.next;
51     }
52 }
53
54 void printOrders()
55 {
56     if (isEmpty())
57     {
58         System.out.println(x: "No orders yet!");
59         return;
60     }
61
62     insertionSortOrders();
63
64     System.out.println(x: "ORDER REPORT (SORTED BY ORDER NAME)");
65
66     NodeOrder tmp = head;
67
68     while (tmp != null)
69     {
70         System.out.println(tmp.data.orderCode + " | " + tmp.data.orderName + " | " + tmp.data.price);
71         tmp = tmp.next;
72     }
73 }
74 }
```



```
1 package com.cm2;
2 import java.util.Scanner;
3
4 public class RoyalDelishSystem
5 {
6     static void MenuOption()
7     {
8         System.out.println(x: "\nROYAL DELISH QUEUE SYSTEM");
9         System.out.println(x: "1. Add Queue");
10        System.out.println(x: "2. Print Queue");
11        System.out.println(x: "3. Remove Queue and Massages");
12        System.out.println(x: "4. Order Report");
13        System.out.println(x: "0. Exit");
14    }
15
16    Run | Debug | Run main | Debug main
17    public static void main(String[] args)
18    {
19        Scanner sc = new Scanner(System.in);
20
21        QueueBuyer queue = new QueueBuyer();
22        OrderList orders = new OrderList();
23
24        int queueCounter = 0;
25
26        int choice;
27
28        do
29        {
30            MenuOption();
31            System.out.print(s: "Select menu: ");
32            choice = sc.nextInt();
33            sc.nextLine();
```

```
34     switch (choice)
35     {
36         case 1:
37             queueCounter++;
38
39             System.out.print(s: "Buyer Name: ");
40             String name = sc.nextLine();
41             System.out.print(s: "Mobile Number: ");
42             String mobile = sc.nextLine();
43
44             Buyer b = new Buyer (queueCounter, name, mobile);
45             queue.enqueue(b);
46             break;
47
48         case 2:
49             queue.printQueue();
50             break;
51
52         case 3:
53             Buyer removed = queue.dequeue();
54
55             if (removed != null)
56             {
57                 System.out.print(s: "Order Code: ");
58                 int code = sc.nextInt();
59                 sc.nextLine();
60                 System.out.print(s: "Order Name: ");
61                 String orderName = sc.nextLine();
62                 System.out.print(s: "Price: ");
63                 int price = sc.nextInt();
64                 sc.nextLine();
65
66                 Order o = new Order(code, orderName, price);
67                 orders.addOrder(o);
68
69                 System.out.println(removed.name + " has ordered " + orderName);
70             }
71             break;
72
73         case 4:
74             orders.printOrders();
75             break;
76
77         case 0:
78             System.out.println(x: "Thank You.");
79             sc.close();
80             break;
81
82         default:
83             System.out.println(x: "Invalid choice! Please Try Again!");
84     }
85 }
86 while (choice != 0);
87 }
88 }
```



```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 1
Buyer Name: Ainra
Mobile Number: 08224500000
Queue successfully added with number: 1

ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 1
Buyer Name: Danra
Mobile Number: 082245511111
Queue successfully added with number: 2

ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 1
Buyer Name: Sanri
Mobile Number: 082245522222
Queue successfully added with number: 3
```



```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 1
Buyer Name: Zaki
Mobile Number: 09876543
Queue successfully added with number: 4

ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 3
Order Code: 123
Order Name: Orange Juice
Price: 25000
Ainra has ordered Orange Juice

ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Messages
4. Order Report
0. Exit
Select menu: 2
List of Buyers in Queue:
2 | Danra | 082245511111
3 | Sanri | 082245522222
4 | Zaki | 09876543
```



```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Massages
4. Order Report
0. Exit
Select menu: 4
ORDER REPORT (SORTED BY ORDER NAME)
123 | Orange Juice | 25000
```

```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Massages
4. Order Report
0. Exit
Select menu: 3
Order Code: 456
Order Name: Apple Juice
Price: 30000
Danra has ordered Apple Juice
```

```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Massages
4. Order Report
0. Exit
Select menu: 2
List of Buyers in Queue:
3 | Sanri | 08224552222
4 | Zaki | 09876543
```

```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Massages
4. Order Report
0. Exit
Select menu: 4
ORDER REPORT (SORTED BY ORDER NAME)
456 | Apple Juice | 30000
123 | Orange Juice | 25000
```

```
ROYAL DELISH QUEUE SYSTEM
1. Add Queue
2. Print Queue
3. Remove Queue and Massages
4. Order Report
0. Exit
Select menu: 0
Thank You.
```

Github:

<https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/cm2>